



Universidad
Rey Juan Carlos

Práctica 2: Memoria

Desarrollo de Juegos con Inteligencia Artificial

Marcos Matutes Rapún, Óscar Rodríguez Marí, Javier Ruibal Piñero

Índice:

Práctica 1:Memoria.....	1
Índice:.....	2
Introducción.....	2
Algoritmo.....	2
Agente.....	3

Introducción

En esta práctica se nos presenta una escena en la que se encuentra nuestro agente, un enemigo y varias murallas a modo de obstáculo. Se nos pide desarrollar un algoritmo de Q-learning para entrenar a nuestro agente para que sobreviva lo máximo posible antes de ser atrapado por el enemigo

Algoritmo

El algoritmo que se ha implementado en para este problema ha sido Q-learning en el cual cada estado está determinado por la posición del agente y la posición del enemigo y las acciones disponibles son moverse una casilla en cada dirección: Norte, Sur, Este y Oeste

Clase QState:

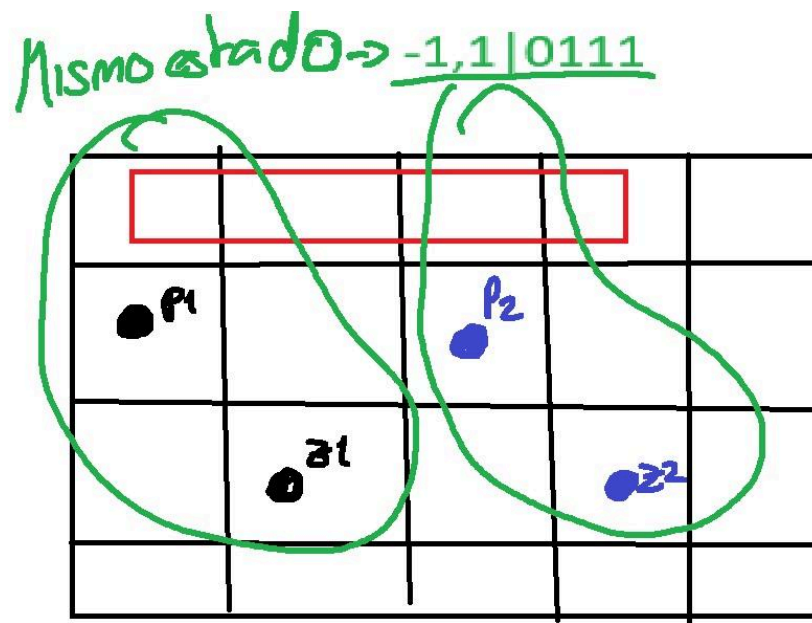
Para que el agente aprenda rápidamente es importante definir correctamente la clase de estados para reducir el número posible de estos. Al principio, se experimentó con una clase simple que almacenaba las posiciones del agente y enemigo pero esto resultaba en una cantidad excesiva de estados posibles y afectaba a la eficacia del algoritmo. Por eso se implementó la siguiente clase:

```
1 using Components;
2 using NavigationDJIA.World;
3 using UnityEngine;
4
5 /// <summary> TODO(alumno): Define el "estado" que usará la Tabla Q para identif ...
6
7
8 namespace GrupoJ
9 {
10     public sealed class QState
11     {
12         public int DistX { get; }
13         public int DistY { get; }
14         public bool WalkUp { get; }
15         public bool WalkDown { get; }
16         public bool WalkRight { get; }
17         public bool WalkLeft { get; }
18
19         public QState(CellInfo agent, CellInfo other, WorldInfo world)
20         {
21             DistX = agent.x - other.x; //Cuanto por encima del zombie
22             DistY = agent.y - other.y; //Cuanto a la derecha del zombie
23
24             WalkUp = IsWalkable(agent.x, agent.y + 1, world); //Si el vecino de arriba es caminable
25             WalkDown = IsWalkable(agent.x, agent.y - 1, world); //Si el vecino de abajo es caminable
26             WalkLeft = IsWalkable(agent.x - 1, agent.y, world); //Si el vecino por la izquierda es caminable
27             WalkRight = IsWalkable(agent.x + 1, agent.y, world); //Si el vecino por la derecha es caminable
28         }
29
30         //Función para calcular si está fuera del mapa
31         private bool IsWalkable(int x, int y, WorldInfo world)
32         {
33             if(x < 0 || x >= world.WorldSize.x || y < 0 || y >= world.WorldSize.y) return false;
34             return world[x, y].Walkable;
35         }
36
37         public string ToKey()
38         {
39             // ejemplo 1,1 | 1111 >>>> 1 encima del zombie 1 a la derecha del zombie y todos los vecinos caminables
40             return $"{DistX},{DistY}|{(WalkUp ? 1 : 0)}{(WalkDown ? 1 : 0)}{(WalkLeft ? 1 : 0)}{(WalkRight ? 1 : 0)}";
41         }
42     }
43 }
```

Las variables DistX y DistY almacenan la posición del agente con respecto al enemigo en ambas coordenadas para reducir el número de casos posibles. Y las variables de Walk evalúan si hay una pared o borde del mapa que impida el desplazamiento hacia alguna dirección para facilitar la toma de decisiones del algoritmo.

Hemos decidido definir cada estado con la posición relativa del agente respecto al zombie y 4 valores representando los vecinos con un 1 si son caminables y un 0 si no,

porque así aunque se sigan creando muchos estados estos son más fáciles de repetirse al no depender de una localización exacta del escenario. Hemos representado un ejemplo de dos posiciones diferentes en el espacio que compartirían el mismo estado.



Clase QMindTrainer

Hemos establecido el learning rate en 0.4, el discount factor en 0.8 y la aleatoriedad de exploración a 0,6 aunque hemos alterado los valores ligeramente durante la etapa de entrenamiento.

Como recompensas y penalizaciones hemos decidido que se reste -100 si es atrapado, -1.5 si se acerca al enemigo, -1 si se queda quieto y -0,5 si dos o más de los vecinos no son caminables, cada paso que sobrevive y se suma +2 si se aleja del enemigo.

Se aplicó el mismo código que en el QMindTester para evitar que el personaje se salga del escenario

```

private float ComputeReward(CellInfo agent, CellInfo other)
{
    float reward = 0f;
    // 1. Penalización máxima si lo atrapan
    if (agent.x == other.x && agent.y == other.y)
        reward = -100f;

    // 2. Cálculo de distancias
    int distActual = Math.Abs(agent.x - other.x) + Math.Abs(agent.y - other.y);
    int distPrevia = Math.Abs(_agentPosition.x - _otherPosition.x) + Math.Abs(_agentPosition.y - _otherPosition.y);

    if (agent.x == _agentPosition.x && agent.y == _agentPosition.y)
    {
        // Penalización por quedarse quieto
        reward = -1.0f;
    }
    else if (distActual >= distPrevia)
    {
        // Recompensa por alejarse
        reward = 2.0f;
    }
    else
    {
        // Penalización leve si se movió hacia el zombie
        reward = -1.5f;
    }

    int count = 0;
    for (int i = -1; i <= 1; i += 2)
    {
        bool walkable1 = !(agent.x + i < 0 && agent.x + i >= _worldInfo.WorldSize.x && agent.y < 0 || agent.y >= _worldInfo.WorldSize.y); //Calcula si los dos vecinos por la izquierda y derecha son caminables
        bool walkable2 = !(agent.x < 0 && agent.x >= _worldInfo.WorldSize.x && agent.y + i < 0 || _agentPosition.y + i >= _worldInfo.WorldSize.y); //Calcula si los dos vecinos por encima y debajo son caminables
        if (walkable1) { count++; }
        if (walkable2) { count++; }
    }

    // Penalización si dos o más de sus vecinos no son caminables
    if (count < 2) { reward = -0.5f; }

    return reward;
}

```

Clase QMindTester

Se realizó un pequeño cambio a la clase QMindTester para evitar que el agente se salga del escenario

```

private CellInfo ApplyAction(CellInfo agentCell, QAction action)
{
    int nx = agentCell.x;
    int ny = agentCell.y;
    switch (action)
    {
        case QAction.Up: ny += 1; break;
        case QAction.Down: ny -= 1; break;
        case QAction.Right: nx += 1; break;
        case QAction.Left: nx -= 1; break;
        case QAction.Stay: return agentCell;
    }

    if (nx >= 0 && nx < _worldInfo.WorldSize.x && ny >= 0 && ny < _worldInfo.WorldSize.y)
    {
        CellInfo targetCell = _worldInfo[nx, ny];
        // Si es caminable se mueve si no devuelve la posición actual.
        if (targetCell.Walkable)
            return targetCell;
    }

    return agentCell;
}

```